

Módulos 2 e 3

Pandas, Visualização e EDA



Curso:

Python para Ciência de Dados



Público:

Iniciantes em programação



Ferramenta:

Google Colab / Jupyter Notebook

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv('dados.csv')
df.head()
```

```
# Visualizar distribuição
sns.histplot(df['idhm'],
             bins=20, kde=True)
plt.show()
```

O que você vai aprender

Dois módulos construídos sobre a base de Python puro do Módulo 1, levando os dados das planilhas até gráficos interativos.

- 01 Pandas: Series e DataFrames
- 02 Leitura e inspeção de CSVs
- 03 Filtragem, seleção e criação de colunas
- 04 Limpeza e qualidade de dados
- 05 Groupby e estatísticas por grupo
- 06 Matplotlib: gráficos de barras e linhas
- 07 Seaborn: scatter, histograma, boxplot
- 08 Mapas de dispersão com lat/long
- 09 EDA: perguntas → gráficos → insights

1. Series e DataFrames

As duas estruturas centrais do Pandas:

Series

Coluna única com índice.
Equivale a um array 1D com rótulos.

DataFrame

Tabela 2D — coleção de Series. Equivale a uma planilha ou tabela SQL.

Comandos essenciais de inspeção:

- **df.shape** → dimensões (linhas, colunas)
- **df.dtypes** → tipo de cada coluna
- **df.describe()** → estatísticas básicas
- **df.head(n)** → primeiras n linhas

LEITURA E INSPEÇÃO INICIAL

```
import pandas as pd

df_escola = pd.read_csv('escola.csv')

print('Linhas e colunas:', df_escola.shape)
print(df_escola.dtypes)
display(df_escola.describe())
```

RESULTADO

Linhas e colunas: (12, 4)

```
nome          object
nota          float64
frequencia    int64
dtype: object
```

2. Seleção, Filtros e Novas Colunas

Operações mais frequentes no dia a dia de análise de dados.

Selecionar colunas

```
df[['nome',  
    'nota']]
```

Filtrar linhas

```
df[df['nota']  
    >= 8]
```

Múltiplas condições

```
df[(df['nota']>  
    =7) &  
    (df['freq']>=75  
    )]
```

Criando novas colunas com apply():

A função apply() percorre cada linha e aplica uma lógica personalizada — equivalente ao MAP ou laço for de Python puro, mas muito mais eficiente.

SELEÇÃO, FILTRO E NOVA COLUNA

```
# Seleção de colunas  
df_escola[['nome', 'nota']]  
  
# Filtro simples  
df_escola[df_escola['nota'] >= 8]  
  
# Filtro composto  
aprovados = df_escola[  
    (df_escola['nota'] >= 7) &  
    (df_escola['frequencia'] >= 75)  
]  
  
# Nova coluna com apply  
df_escola['situacao'] = df_escola.apply(  
    lambda row: 'Aprovado'  
        if row['nota'] >= 7  
        and row['frequencia'] >= 75  
    else 'Reprovado',  
    axis=1  
)
```

3. Limpeza de Dados

Mesmo datasets 'prontos' precisam passar por uma checagem de qualidade antes de qualquer análise.

✓	Valores ausentes	<code>df.isna().sum()</code>
✓	Linhas duplicadas	<code>df.duplicated().sum()</code>
✓	Tipos incorretos	<code>df.dtypes</code>
✓	Renomear colunas	<code>df.rename(columns={...})</code>

LIMPEZA E PADRONIZAÇÃO

```
# Verificações básicas de qualidade
print('Valores ausentes por coluna:')
print(df_escola.isna().sum())
print('Linhas duplicadas:',
      df_escola.duplicated().sum())

# Renomear colunas do dataset de cidades
cidades = df_cidades.rename(columns={
    'CITY': 'cidade',
    'STATE': 'uf',
    'ESTIMATED_POP': 'populacao',
    'GDP_CAPITA': 'pib_per_capita',
    'IDHM': 'idhm'
})

# Preencher valores ausentes
cidades['idhm'].fillna(
    cidades['idhm'].median(),
    inplace=True
)
```

4. Groupby e Estatísticas por Grupo

groupby() é o equivalente ao SQL GROUP BY — agrupa linhas por categoria e calcula métricas por grupo.

situacao	nota (média)	frequência (média)
Aprovado	8.23	87.5
Reprovado	5.14	63.2

nlargest() — ranking rápido

```
idades.nlargest(10, 'populacao')  
[['cidade', 'uf', 'populacao']]
```

GROUPBY E AGREGAÇÕES

```
# Estatísticas por situação  
df_escola.groupby('situacao')[  
    ['nota', 'frequencia']  
].mean().round(2)  
  
# IDHM médio por estado  
idhm_uf = (  
    cidades.groupby('uf')['idhm']  
    .mean()  
    .sort_values(ascending=False)  
    .round(4)  
)  
idhm_uf.head(10)  
  
# Top 10 cidades mais populosas  
mais_pop = cidades.nlargest(  
    10, 'populacao_estimada'  
)[['cidade', 'uf', 'populacao_estimada']]  
  
# Múltiplas agregações  
cidades.groupby('capital').agg(  
    media_idhm=('idhm', 'mean'),  
    total_hoteis=('hoteis', 'sum')  
) .round(3)
```

5. Matplotlib: Gráficos Essenciais

Matplotlib oferece controle total sobre cada elemento do gráfico.

Barras

Comparar categorias

Linhas

Evolução temporal

Scatter

Relação entre vars.

Hist

Distribuição de dados

Anatomia de um gráfico Matplotlib:

```
plt.figure(figsize=(10, 4)) → cria o canvas  
plt.title() / xlabel() / ylabel() → rótulos  
plt.show() → exibe o gráfico
```

MATPLOTLIB + SEABORN

```
import matplotlib.pyplot as plt  
import seaborn as sns  
  
sns.set_theme(style='whitegrid')  
  
# Gráfico de barras: notas por aluno  
plt.figure(figsize=(10, 4))  
plt.bar(  
    df_escola['nome'],  
    df_escola['nota'],  
    color='steelblue'  
)  
plt.title('Nota por aluno')  
plt.xlabel('Aluno')  
plt.ylabel('Nota')  
plt.xticks(rotation=45)  
plt.show()  
  
# Relação nota vs frequência  
plt.figure(figsize=(6, 4))  
sns.scatterplot(  
    data=df_escola,  
    x='frequencia',  
    y='nota',  
    s=100  
)  
plt.show()
```

6. Seaborn: Visualizações Estatísticas

barplot()

Barras com intervalo de confiança automático

histplot()

Histograma com KDE para distribuições

scatterplot()

💡 Seaborn vs Matplotlib

Use Seaborn para gráficos estatísticos rápidos e bonitos; use Matplotlib quando precisar de controle fino sobre posicionamento, anotações e layout composto.

SEABORN NA PRÁTICA

```
# Top 10 cidades por população
top_pop = cidades.nlargest(10,
                           'ESTIMATED_POP')

sns.barplot(
    data=top_pop,
    x='CITY', y='ESTIMATED_POP',
    palette='crest'
)
plt.xticks(rotation=45)
plt.title('10 cidades mais populosas')
plt.show()

# Distribuição do IDHM
sns.histplot(
    cidades['IDHM'],
    bins=20,
    kde=True,
    color='darkorange'
)

# PIB per capita x IDHM com grupos
sns.scatterplot(
    data=cidades,
    x='GDP_CAPITA', y='IDHM',
    hue='CAPITAL',
    palette='Set1'
)
plt.show()
```


7. Mapa de Dispersão com Lat/Long

Sem bibliotecas geoespaciais extras, já é possível criar um mapa simplificado usando longitude e latitude como eixos X e Y de um scatter.

c=cidades['IDHM']

Cor representa o desenvolvimento humano

s=pop / 20000

Tamanho do ponto $\propto$ população estimada

cmap='viridis'

Paleta de cores contínua e perceptualmente uniforme

MAPA DE DISPERSÃO (LAT/LONG)

```
# Mapa de dispersão: cada ponto = cidade
plt.figure(figsize=(10, 8))

scatter = plt.scatter(
    cidades['LONG'],
    cidades['LAT'],
    c=cidades['IDHM'],
    s=cidades['ESTIMATED_POP'] / 20000,
    cmap='viridis',
    alpha=0.6,
    edgecolors='none'
)

# Barra de cor: legenda do IDHM
plt.colorbar(scatter,
             label='IDHM')

plt.title(
    'Distribuição espacial das cidades\n'
    '(cor = IDHM | tamanho = população)'
)
plt.xlabel('Longitude')
plt.ylabel('Latitude')
plt.tight_layout()
plt.show()
```

8. EDA: Perguntas → Gráficos → Insights

A Análise Exploratória de Dados (EDA) não é sobre rodar gráficos aleatórios — é sobre formular perguntas e escolher a visualização certa para respondê-las.

P1: Desempenho acompanha frequência?

 `scatterplot + hue='situacao'`

💡 Aprovados se concentram no canto superior direito.

P2: Capitais têm IDHM mais alto?

 `barplot + groupby('capital')`

💡 Capitais têm média de IDHM superior às demais.

P3: Existe relação PIB × IDHM?

 `scatterplot + regressão visual`

💡 Tendência positiva, mas com alta dispersão.

EDA NA PRÁTICA

```
# EDA 1: notas vs frequência por situação
escola['situacao'] = escola.apply(
    lambda r: 'Aprovado'
        if r['nota'] >= 7 and r['frequencia'] >= 75
        else 'Reprovado', axis=1
)
sns.scatterplot(
    data=escola,
    x='frequencia', y='nota',
    hue='situacao', s=120
)
# Anotar nomes no gráfico
for _, row in escola.iterrows():
    plt.text(row['frequencia']+0.5,
             row['nota']+0.03,
             row['nome'], fontsize=9)

plt.show()

# EDA 2: IDHM capital vs não-capital
idhm_cap = eda.groupby(
    'capital', as_index=False
)['idhm'].mean()
sns.barplot(
    data=idhm_cap,
    x='capital', y='idhm',
    palette='viridis'
)
plt.show()
```

Conclusão dos Módulos 2 e 3

Tópico	O que aprendemos
Pandas — Estruturas	Series, DataFrame, read_csv(), head(), describe()
Filtros e seleção	Indexação booleana, operadores & e , colunas
Criação de colunas	apply(), lambda, transformações derivadas
Limpeza	isna(), duplicated(), rename(), fillna()
Agrupamento	groupby(), agg(), nlargest(), sort_values()
Matplotlib	bar(), scatter(), hist(), figure(), show()
Seaborn	barplot(), histplot(), scatterplot(), hue=
EDA	Perguntas → visualizações → insights interpretados

🔗 **Pratique: resolva os exercícios dos Módulos 2 e 3 antes de avançar!**

[Disponível no NOSSO GITHUB](#)

Próximo Passo: Módulo 4

Machine Learning com Scikit-learn